P1099R4
Using Enum

Published Proposal, 2019-03-10

This version:
https://github.com/atomgalaxy/using-enum/using-enum.bs

Authors:
Gašper Ažman <gasper.azman@gmail.com>
Jonathan Müller <jonathan.mueller@nathan.net>

Audience:
CWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

Class enums are restricted namespaces. Let’s extend the using declaration to them.

Table of Contents

§ 1. Revision History

- r1: Typos. Changed target from LEWG to EWG.
- r2: Added FAQ and wording, and extended to regular enums.
- r3: Added examples. Nothing of substance changed.
- r4: Clarified that `using enum enum_name` is supposed to have semantics as if the names were declared in the local scope as opposed to the way *using-directive* does it, as per EWG poll. Added Eric Niebler's example for using this feature to enable ADL-only functions on exposed enumerators (to support `std::begin` being a CPO). Added editorial notes for renaming the current Using Directive to Using Namespace Directive on Richard Smith's request.

§ 2. Status of this paper

This paper has been approved by EWG in Kona 2019 and sent to Core with the ship vehicle of C++20.

§ 3. Motivation

> The single biggest deterrent to use of scoped enumerations is the inability to associate them with a *using directive*.
>
> — Dan Saks

Consider an enum class:

```
enum class rgba_color_channel { red, green, blue, alpha };
```

Currently, a switch using this enum looks as follows:

```
std::string_view to_string(rgba_color_channel channel) {
  switch (channel) {
    case rgba_color_channel::red:   return "red";
    case rgba_color_channel::green: return "green";
    case rgba_color_channel::blue:  return "blue";
    case rgba_color_channel::alpha: return "alpha";
  }
}
```

The necessary repetition of the `enum class` name reduces legibility by introducing noise in contexts where said name is obvious.

To eliminate the noise penalty for introducing long (but descriptive) `enum class` names, this paper proposes that the statement

```
using enum rgba_color_channel;
```

introduce the enumerator identifiers into the local scope, so they may be referred to unqualified.

Furthermore, the syntax

```
using rgba_color_channel::red;
```

should bring the identifier `red` into the local scope, so it may be used unqualified.

The above example would then be written as

```
std::string_view to_string(rgba_color_channel channel) {
  switch (my_channel) {
    using enum rgba_color_channel;
    case red:   return "red";
    case green: return "green";
    case blue:  return "blue";
    case alpha: return "alpha";
  }
}
```

§ 4. Rationale

§ 4.1. Consistency

`enum class`es and `enum`s are not classes - they are closer to namespaces comprising `static constexpr` inline variables. The familiar `using` syntax that works for namespaces should therefore apply to them as well, in some fashion. Because they are closed, small, and do not contain overload sets, we can do better than the *using-directive* does for namespaces, and actually get the identifiers into the local scope, which is what the user expects.

§ 4.2. Better Identifiers

The introduction of this feature would allow better naming of enumerations. Currently, enums are named with as short an identifier as possible, often to the point of absurdity, when they are reduced to completely nondescriptive abbreviations that only hint at their proper meaning. (Just what does `zfqc::add_op` really mean?)

With this feature, identifiers become available to unqualified lookup in local contexts where their source is obvious, giving control of lookup style back to the user of the enum, instead of baking lookup semantics into the type of the enum.

§ 4.3. Evidence of Need

At a casual search, we were able to locate this thread on stackoverflow.

Anecdotally, 100% of people the authors have shown this to (~30) at CppCon have displayed a very enthusiastic response, with frequent comments of "I'd use enum classes but they are too verbose, this solves my problem!"

§ 5. Proposal

§ 5.1. Syntax: `using ENUM_ID::IDENTIFIER`

We propose to allow the syntax of

```
using ENUM_ID::IDENTIFIER
```

to introduce the `IDENTIFIER` into the local namespace, aliasing `ENUM_ID::IDENTIFIER`.

This would mirror the current syntax for introducing namespaced names into the current scope.

> Note: this does not conflict with [P0945R0], because that paper only deals with the syntax `using name = id-expression`, which duplicates the enumerator name.

§ 5.2. Syntax: `using enum IDENTIFIER`

We propose the addition of a new `using enum` statement:

```
using enum IDENTIFIER;
```

This makes all the enumerators of the enum available for lookup in the local scope. It's almost as if it expanded to a series of `using ENUM::ENUMERATOR` statements for every enumerator in the enum, but doesn't actually introduce any declarations into the local scope.

(Note: this was changed from "works as a using-directive" to the current way with a strong direction poll from EWG.)

§ 6. Examples

§ 6.1. Strongly typed enums with global identifiers

This proposal lets you make strongly-typed enums still export their identifiers to namespace scope, therefore behaving like the old enums in that respect:

```
namespace my_lib {

enum class errcode {
  SUCCESS = 0,
  ENOMEM = 1,
  EAGAIN = 2,
  ETOOSLOW = 3
};
using enum errcode;

}

namespace  {

my_lib::errcode get_widget() {
  using namespace my_lib;
  return ETOOSLOW; // works, and conversions to int don’t.
}

}
```

§ 6.2. Switching with no syntax overhead

The proposal allows for importing enums inside the switch body, which is a scope, and using them for labels:

```
enum class rgba_color_channel { red, green, blue, alpha};

std::string_view to_string(rgba_color_channel channel) {
  switch (my_channel) {
    using enum rgba_color_channel;
    case red:   return "red";
    case green: return "green";
    case blue:  return "blue";
    case alpha: return "alpha";
  }
}
```

§ 6.3. Adding ADL-only Functions to Enumerations:

The proposal allows for adding ADL-only functions to enumerations without enumerators (supported now) and enumerators (currently not supported):

```
namespace ns {
  struct E_detail {
    enum E {
      e1
    };
    friend void swap(E&, E&);  // adl-only swap in the only associated scop
  };
  using E = E_detail::E;  // import E into ns
  using enum E;           // expose the enumerators of E in ns. Also note t
}

int main() {
  auto x = ns::e1;
  auto y = ns::e2;
  swap(x, y); // finds the swap in the associated struct
}
```

This example was slightly modified from Eric Niebler's on the *lib* mailing list when trying to find a way to make `std::begin` and `std::end` CPOs in a backwards-compatible fashion.

§ 7. Frequently Asked Questions

§ 7.1. Has this been implemented?

Yes. The author has an implementation in clang. It has not been reviewed or released yet, however. There do not seem to be major issues with implementation. In particular, the `using ENUM::IDENTIFIER` syntax literally entailed removing a condition from an if-statement, and that was it.

§ 7.2. Can I do this with unscoped enums?

Yes. The motivation for that is the pattern

```
class foo {
   enum bar {
     A,
     B,
     C
   };
};
```

which was superceeded by scoped enums. With the feature this paper proposes one can bring `A`, `B` and `C` into the local scope by invoking:

```
using enum ::foo::bar;
```

§ 7.3. Are you proposing mirroring the namespace alias syntax as well?

No. We already have a way to do that, and it looks like this:

```
using my_alias = my::name_space::enum_name;
```

In addition, [P0945R0] proposes deprecating namespace aliases in favor of generalized `using name = id_expression`, so doing this would go counter the current movement of the standard.

§ 7.4. Why not allow `using enum struct/class ENUM_ID;`?

Because would have been a needless complication and would introduce another layer of "`struct` and `class` don't match" linter errors that current `class`es and `struct`s already have with forward declarations.

§ 7.5. Why propose `using ENUM_ID::IDENTIFIER` at all?

... given that the following already works:

```
constexpr auto red = rgba_color_channel::red;
```

and that, given [P0945R0], this will work:

```
using red = rgba_color_channel::red;
```

The reason is "DRY" - don't repeat yourself - one is forced to repeat the name of the enumerator. That said, the authors are perfectly willing to throw this part of the paper out if the `using enum ENUM_ID` piece gets consensus and this is the stumbling block.

§ 8. Proposed Wording

§ 8.1. Preface

The idea is that the identifiers appear as if they were declared in the declarative region where the using-enum-directive appears, and not model the using-directive's "enclosing namespace" wording.

All wording is relative to the working draft of the ISO/IEC IS 14882: N4765, though, as it is almost entirely additive, it is also a valid diff to N8000.

§ 8.2. Changes

In chapter [namespace.udecl]:

3. In a *using-declaration* used as a *member-declaration*, each *using-declarator* ~~'s~~ shall either name an enumerator or have a *nested-name-specifier* ~~shall name~~ naming a base class of the class being defined. [Note: this exception allows the introduction of enumerators into class scope. --end note]

~~7. A *using-declaration* shall not name a scoped enumerator.~~

8. A *using-declaration* that names a class member that is not an enumerator shall be a *member-declaration*. [Note: the exception for enumerators allows the introduction of class members that are enumerators into non-class scope --end note]

In chapter [dcl.dcl], in [dcl.enum], add section titled "Using Enum Directive", with the stable reference "[enum.udir]".

using-enum-directive:
attribute-specifier-seq$_{opt}$ `using` *elaborated-type-specifier*;

1. The *elaborated-type-specifier* shall name an enumeration. [Note: an elaborated type specifier for an enumeration always begins with `enum` -- end note]
2. The *elaborated-type-specifier* shall not name a dependent type.
3. The optional *attribute-specifier-seq* appertains to the *using-enum-directive*.
4. A *using-enum-directive* introduces the enumerator names of the named enumeration into the scope in which it appears, as synonyms for the enumerators of the named enumeration. [Note: this means that they may be used, qualified or unqualified, after the *using-enum-directive* -- end note]

[Note: an *using-enum-directive* in class scope adds the enumerators of the named enumeration as members to the scope. This means they are accessible for member lookup. Example:

```
enum class E { e1, e2 };
struct S {
  using enum E; // introduces e1 and e2 into S
};
void f() {
  S s;
  s.e1; // well-formed, names E::e1.
  S::e2; // well-formed, names E::e2.
}
```

-- end note]

Under [basic.def], add (just after *using-directive*) (and renumber section):

2.17. — it is a *using-enum-directive*

In [gram.dcl], under *block-declaration*:

block-declaration
[...]
using-directive
using-enum-directive

In [class.mem], under *member-declaration*:

member-declaration
[...]
using-declaration
using-enum-directive

Note to editor:

For greater consistency, rename, everywhere:

- "using directive" to "using namespace directive"
- and "*using-directive*" to "*using-namespace-directive*"
- [namespace.udir]'s title should be Using Namespace Directive.

§ 9. Acknowledgements

The authors would like to thank Marcel Ebmer and Lisa Lippincott for early feedback, and the members of the BSI C++ WG for further feedback, especially Graham Haynes and Barry Revzin. Even further feedback was provided by Tomas Puverle, who encouraged us to extend it to `enum`s, and Dan Saks for the permission to include a quotation from him. A special thanks to Jeff Snyder who presented this to EWG, and to Richard Smith for pointing out direct injection is really the better way to go. Casey Carter pointed out some issues with wording typography, and realized that this paper allows adding adl-only functions to enumerators.

Another big thank-you to the Wizard of Worde, Richard Smith, who helped with the final wording, and found the need to make enum names non-dependent.

§ References

§ Informative References

[P0945R0]

Richard Smith. Generalizing alias declarations. 10 February 2018. URL: https://wg21.link/p0945r0